imageprocessing

May 16, 2026

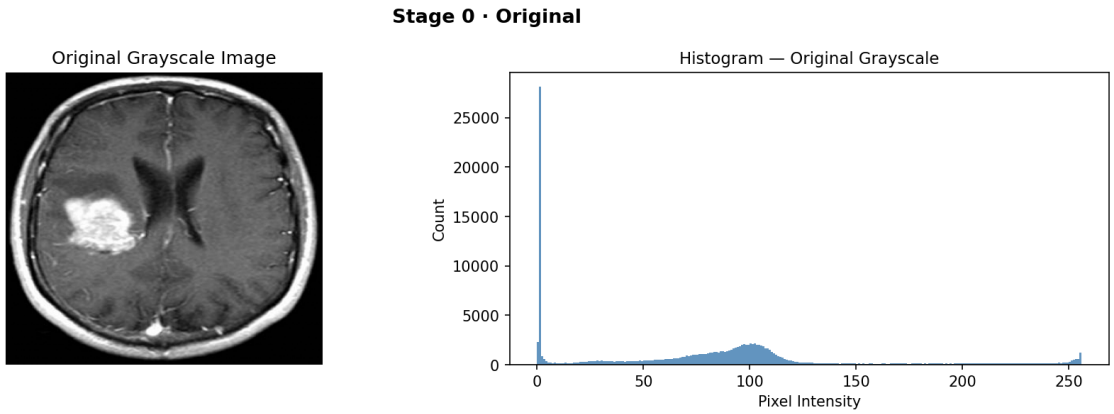
[13]:
```
# Cell 1 - Imports
from io import BytesIO
import cv2
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from skimage.metrics import peak_signal_noise_ratio, mean_squared_error,␣
 ↪structural_similarity
import numpy as np

#    Global figure defaults
plt.rcParams.update({
    "figure.dpi"      : 150,
    "axes.titlesize"  : 11,
    "axes.labelsize"  : 10,
    "figure.facecolor": "white",
})
IMAGE_DISPLAY_SIZE = (5, 5)   # single image panel (w, h) in inches
HIST_SIZE          = (7, 4)   # standalone histogram panel
```

[14]:
```
# Cell 2 - Load Image
IMAGE_PATH = "./test_images/yes/Y11.jpg"

img_gray_pil = Image.open(IMAGE_PATH).convert("L")
img_arr      = np.array(img_gray_pil, dtype=np.float32) / 255.0
img_u8       = (img_arr * 255).astype(np.uint8)
gray         = img_u8

print(f"Shape : {gray.shape}  |  dtype : {gray.dtype}  |  range : [{gray.
 ↪min()}, {gray.max()}]")
```

```
Shape : (369, 400)  |  dtype : uint8  |  range : [0, 255]
```

[15]:
```
# Cell 2a - Original Image (standalone)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].imshow(img_u8, cmap='gray')  # use colormap!
axes[0].set_title("Original Grayscale Image", fontsize=12)
```

```
axes[0].axis("off")
axes[1].hist(gray.ravel(), bins=256, range=(0,256), color="steelblue", alpha=0.
↪85)
axes[1].set_title("Histogram - Original Grayscale")
axes[1].set_xlabel("Pixel Intensity")
axes[1].set_ylabel("Count")
plt.suptitle("Stage 0 · Original", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("original_with_histogram.png", dpi=300, bbox_inches='tight')

plt.show()
print(f"Shape : {img_u8.shape}  |  dtype : {img_u8.dtype}  |  range : [{img_u8.
↪min()}, {img_u8.max()}]")
```

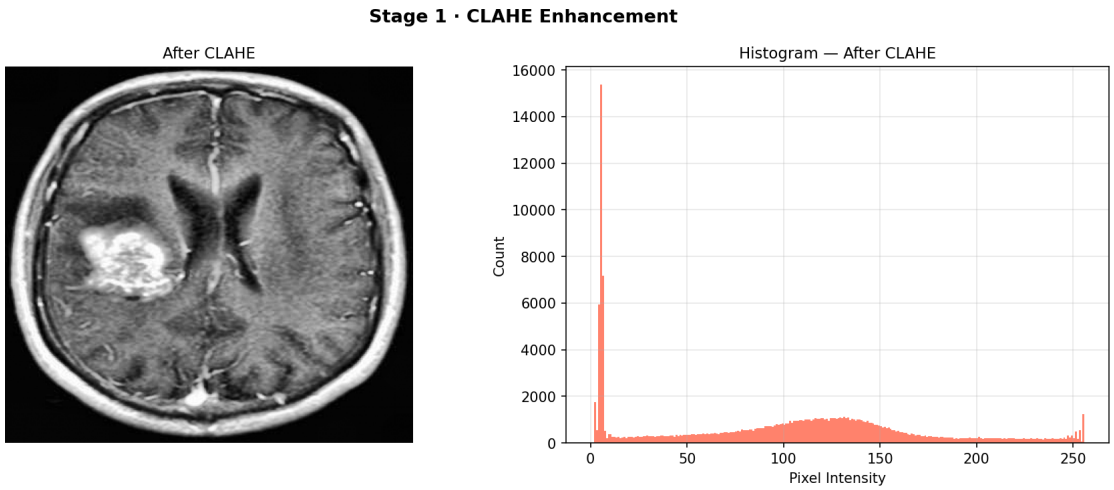



```
Shape : (369, 400)  |  dtype : uint8  |  range : [0, 255]
```

[16]:
```
# Cell 3 - Stage 1 : CLAHE Enhancement
clahe      = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
gray_clahe = clahe.apply(gray)
```

[17]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# CLAHE Enhanced Image
axes[0].imshow(gray_clahe, cmap="gray")
axes[0].set_title("After CLAHE")
axes[0].axis("off")

# CLAHE Histogram Only
axes[1].hist(
    gray_clahe.ravel(),
    bins=256,
    range=(0, 256),
```

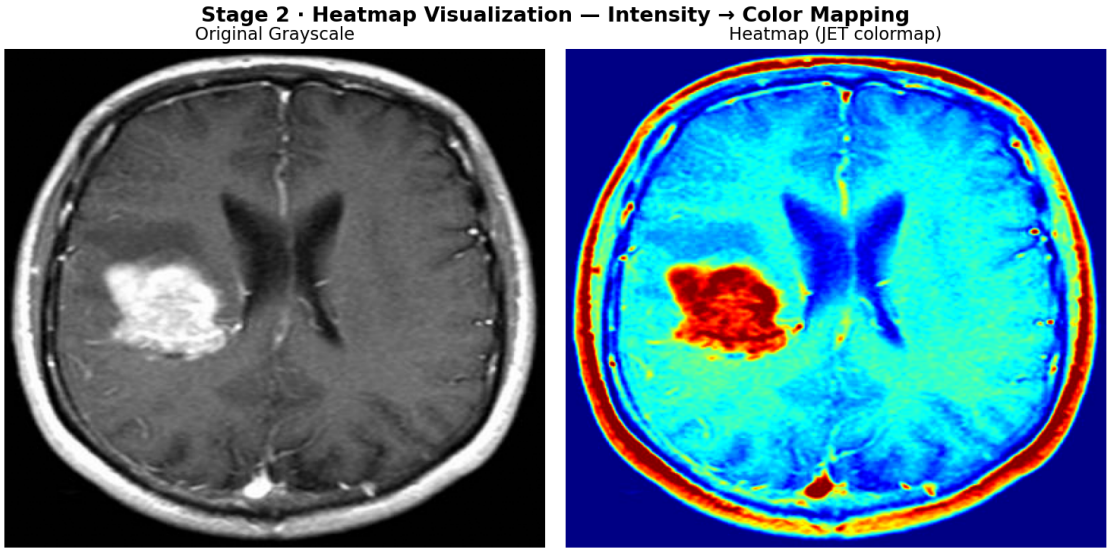
```
    color="tomato",
    alpha=0.8
)

axes[1].set_title("Histogram - After CLAHE")
axes[1].set_xlabel("Pixel Intensity")
axes[1].set_ylabel("Count")
axes[1].grid(alpha=0.3)

plt.suptitle("Stage 1 · CLAHE Enhancement", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("clahe_with_histogram.png", dpi=300, bbox_inches='tight')
plt.show()
```



[18]:
```
# Cell 4 - Stage 2 : Heatmap (JET colormap)
heatmap_bgr = cv2.applyColorMap(gray, cv2.COLORMAP_JET)
heatmap_rgb = cv2.cvtColor(heatmap_bgr, cv2.COLOR_BGR2RGB)
```

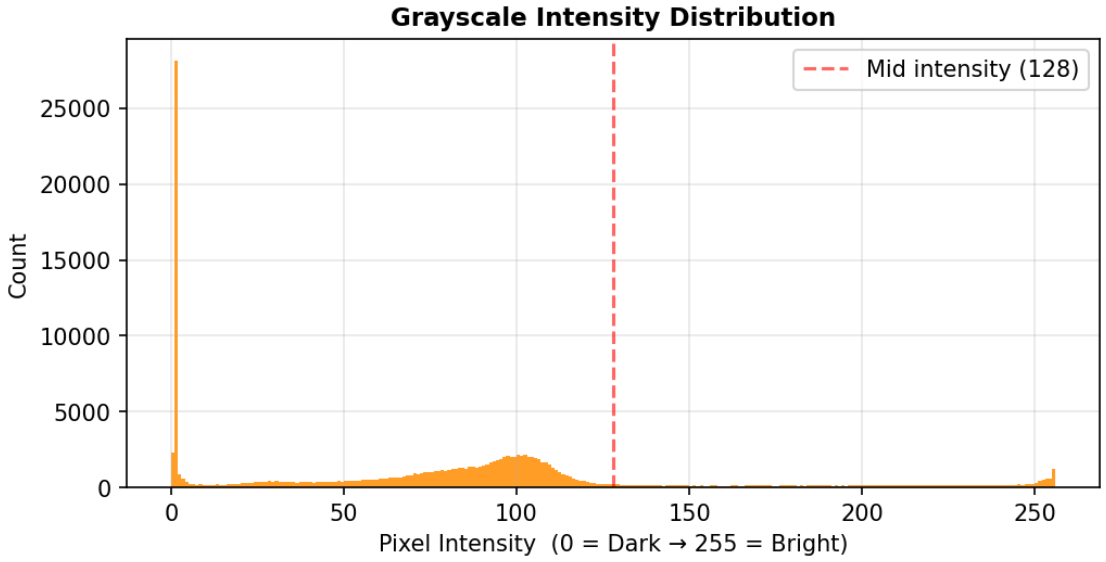
[19]:
```
# Cell 4a - Heatmap : Image Comparison
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(gray,        cmap="gray")
axes[0].set_title("Original Grayscale")
axes[0].axis("off")

axes[1].imshow(heatmap_rgb)
axes[1].set_title("Heatmap (JET colormap)")
axes[1].axis("off")

plt.suptitle("Stage 2 · Heatmap Visualization - Intensity → Color Mapping",
```

```
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("heatmap_comparison.png", dpi=300, bbox_inches="tight")
plt.show()
```



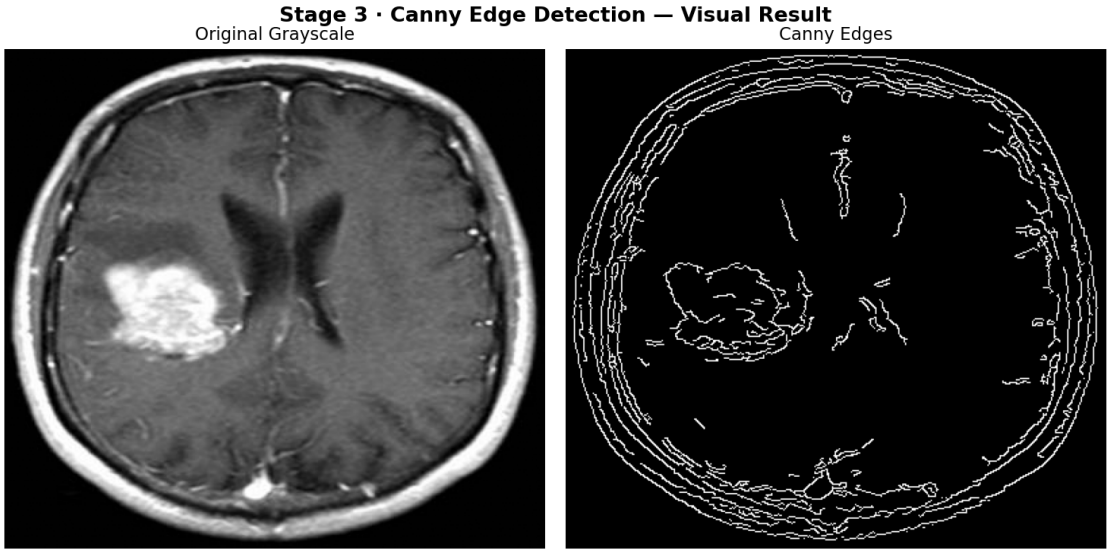
[20]:
```
# Cell 4b - Heatmap : Grayscale Intensity Distribution (standalone)
fig, ax = plt.subplots(figsize=HIST_SIZE)

ax.hist(gray.ravel(), bins=256, range=(0, 256), color="darkorange", alpha=0.85)
ax.axvline(x=128, color="red", linestyle="--", alpha=0.6, label="Mid intensity␣
 ↪(128)")
ax.set_title("Grayscale Intensity Distribution", fontweight="bold")
ax.set_xlabel("Pixel Intensity  (0 = Dark → 255 = Bright)")
ax.set_ylabel("Count")
ax.legend()
ax.grid(alpha=0.3)

plt.suptitle("Stage 2 · Heatmap - Underlying Intensity Distribution",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("heatmap_histogram.png", dpi=300, bbox_inches="tight")
plt.show()
```

Stage 2 · Heatmap — Underlying Intensity Distribution

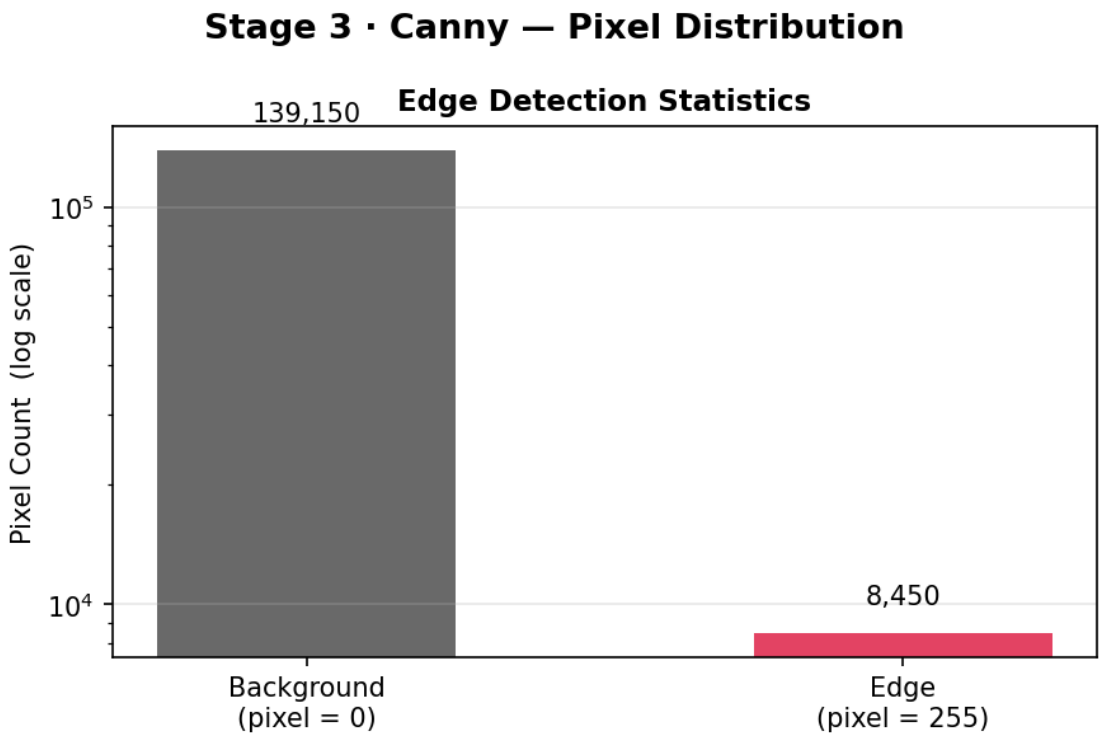


[21]:
```python
# Cell 5 - Stage 3 : Canny Edge Detection
edges = cv2.Canny(gray, 100, 200)

edge_pixel_count  = int(np.sum(edges == 255))
background_pixels = int(np.sum(edges == 0))
edge_percentage   = edge_pixel_count / edges.size * 100
```

[43]:
```python
# Cell 5a - Canny : Image Comparison
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(gray,  cmap="gray")
axes[0].set_title("Original Grayscale")
axes[0].axis("off")

axes[1].imshow(edges, cmap="gray")
axes[1].set_title(
    f"Canny Edges"
)
axes[1].axis("off")

plt.suptitle("Stage 3 · Canny Edge Detection - Visual Result",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("canny_comparison.png", dpi=300, bbox_inches="tight")
plt.show()
```



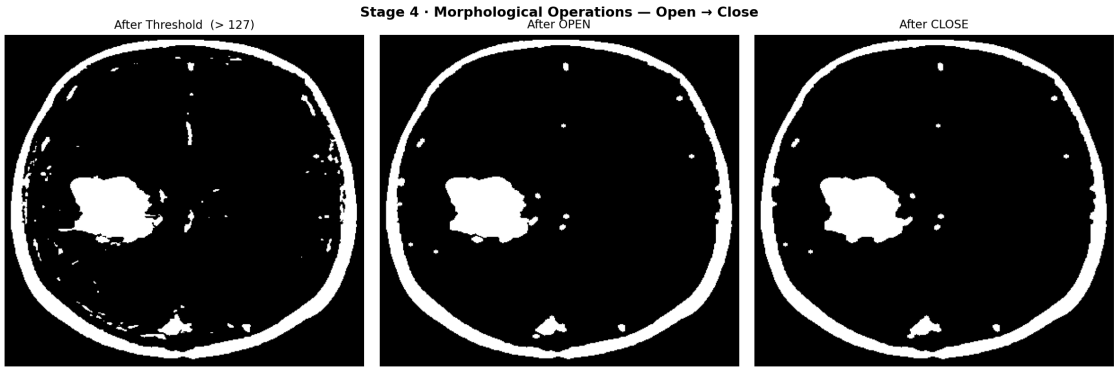
[23]:
```
# Cell 5b - Canny : Edge Statistics (standalone bar chart)
fig, ax = plt.subplots(figsize=(6, 4))

bars = ax.bar(
    ["Background\n(pixel = 0)", "Edge\n(pixel = 255)"],
    [background_pixels, edge_pixel_count],
    color=["#444444", "crimson"],
    alpha=0.80,
    width=0.5,
)
ax.set_yscale("log")
ax.set_ylabel("Pixel Count  (log scale)")
ax.set_title("Edge Detection Statistics", fontweight="bold")

for bar, val in zip(bars, [background_pixels, edge_pixel_count]):
    ax.text(bar.get_x() + bar.get_width() / 2,
            val * 1.15, f"{val:,}",
            ha="center", va="bottom", fontsize=10)

ax.grid(axis="y", alpha=0.3)
plt.suptitle("Stage 3 · Canny - Pixel Distribution",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("canny_statistics.png", dpi=300, bbox_inches="tight")
plt.show()
```



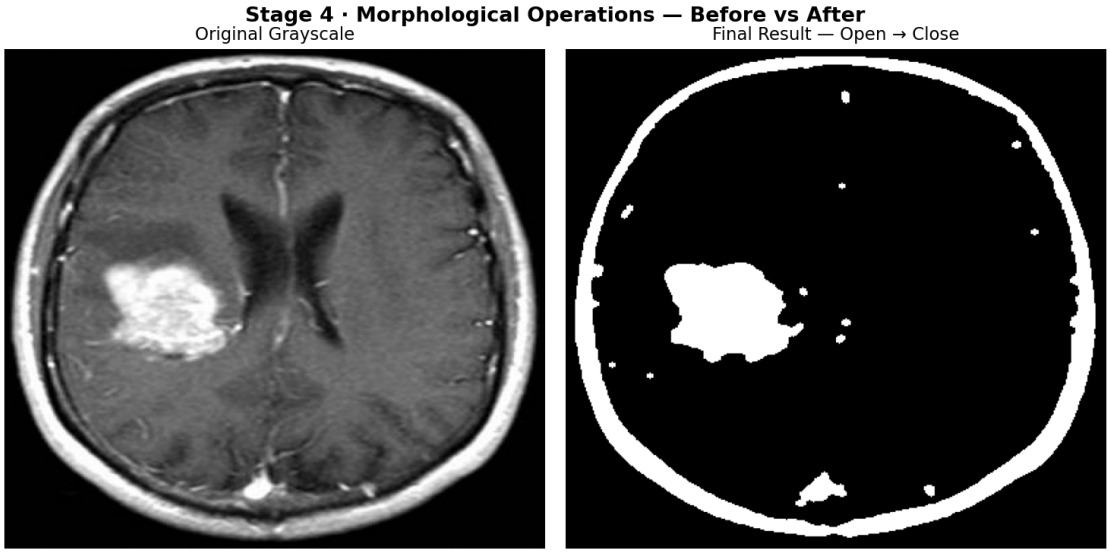
[24]:
```
# Cell 6 - Stage 4 : Morphological Open → Close
_, thresh_morph = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
kernel          = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
after_open      = cv2.morphologyEx(thresh_morph, cv2.MORPH_OPEN,  kernel)
after_close     = cv2.morphologyEx(after_open,   cv2.MORPH_CLOSE, kernel)
```

[44]:
```
# Cell 6a - Morphology : Step-by-Step Comparison
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(thresh_morph, cmap="gray")
axes[0].set_title("After Threshold  (> 127)")
axes[0].axis("off")

axes[1].imshow(after_open,   cmap="gray")
axes[1].set_title("After OPEN")
axes[1].axis("off")

axes[2].imshow(after_close,  cmap="gray")
axes[2].set_title("After CLOSE")
axes[2].axis("off")

plt.suptitle("Stage 4 · Morphological Operations - Open → Close",
             fontsize=13, fontweight="bold")
```

```
plt.tight_layout()
plt.savefig("morphological_steps.png", dpi=300, bbox_inches="tight")
plt.show()
```



[45]:
```
# Cell 6b - Morphology : Before vs Final (compact 2-panel)
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(gray,        cmap="gray")
axes[0].set_title("Original Grayscale")
axes[0].axis("off")

axes[1].imshow(after_close, cmap="gray")
axes[1].set_title("Final Result - Open → Close")
axes[1].axis("off")

plt.suptitle("Stage 4 · Morphological Operations - Before vs After",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("morphological_before_after.png", dpi=300, bbox_inches="tight")
plt.show()
```

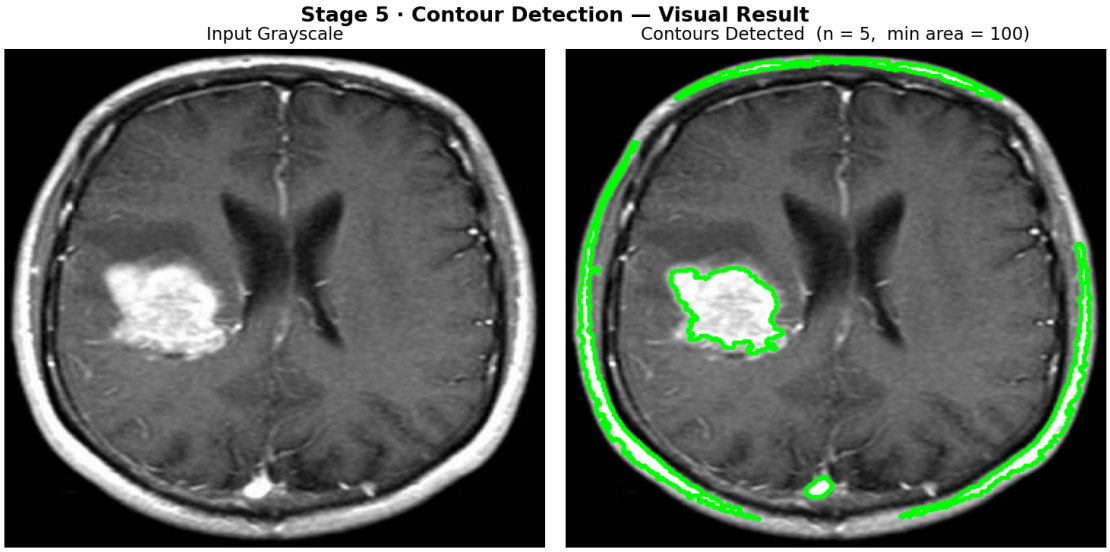


[27]:
```
# Cell 7 - Stage 5 : Contour Detection
_, thresh_cnt   = cv2.threshold(gray, 210, 255, cv2.THRESH_BINARY)
contours, _     = cv2.findContours(thresh_cnt, cv2.RETR_TREE, cv2.
 ↪CHAIN_APPROX_SIMPLE)
contours_sorted = sorted(contours, key=cv2.contourArea, reverse=True)
filtered        = [c for c in contours_sorted if cv2.contourArea(c) > 100]

contour_canvas  = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
cv2.drawContours(contour_canvas, filtered, -1, (0, 255, 0), 2)

print(f"Total contours found : {len(filtered)}")
```

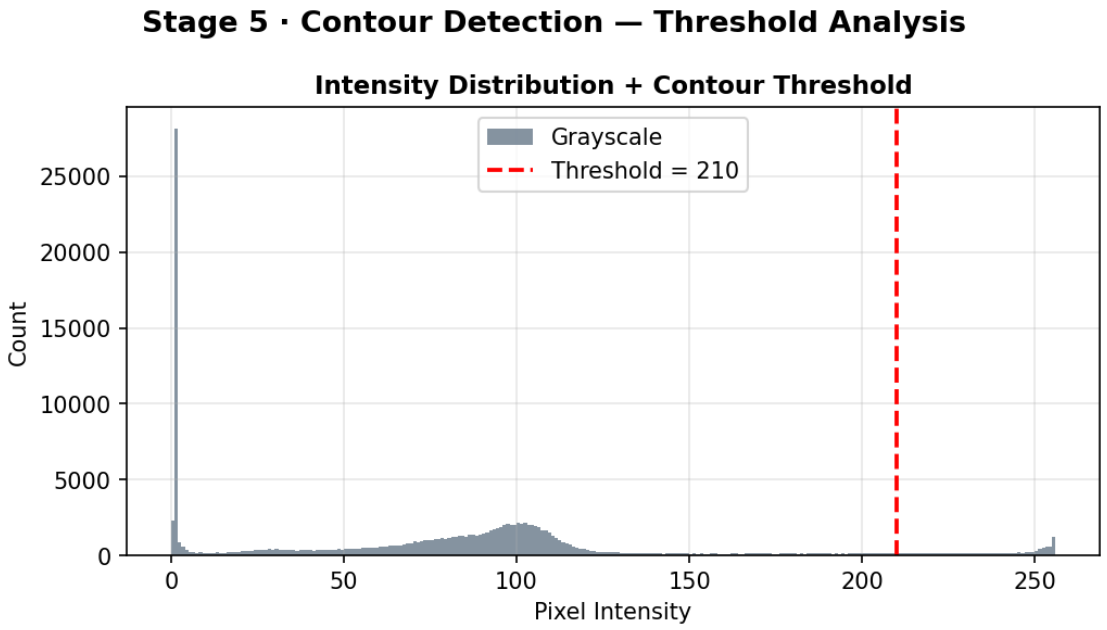
```
Total contours found : 5
```

[28]:
```
# Cell 7a - Contour : Image Comparison
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(gray,           cmap="gray")
axes[0].set_title("Input Grayscale")
axes[0].axis("off")

axes[1].imshow(contour_canvas)
axes[1].set_title(f"Contours Detected  (n = {len(filtered)},  min area = 100)")
axes[1].axis("off")

plt.suptitle("Stage 5 · Contour Detection - Visual Result",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("contour_comparison.png", dpi=300, bbox_inches="tight")
```

```
plt.show()
```



[29]:
```
# Cell 7b - Contour : Threshold Histogram (standalone)
fig, ax = plt.subplots(figsize=HIST_SIZE)

ax.hist(gray.ravel(), bins=256, range=(0, 256),
        color="slategray", alpha=0.85, label="Grayscale")
ax.axvline(210, color="red", linewidth=1.8,
           linestyle="--", label="Threshold = 210")
ax.set_title("Intensity Distribution + Contour Threshold", fontweight="bold")
ax.set_xlabel("Pixel Intensity")
ax.set_ylabel("Count")
ax.legend()
ax.grid(alpha=0.3)

plt.suptitle("Stage 5 · Contour Detection - Threshold Analysis",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("contour_histogram.png", dpi=300, bbox_inches="tight")
plt.show()
```

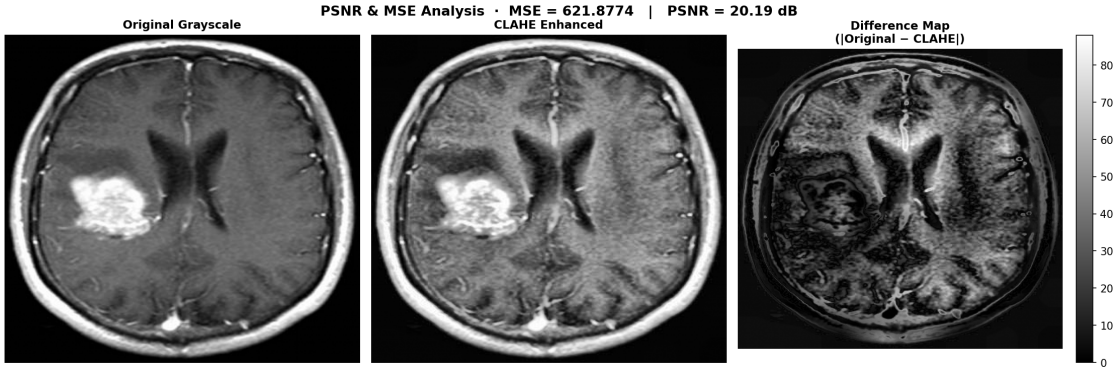


[30]:
```
# Cell 8 - PSNR & MSE Analysis (Original vs CLAHE)
from skimage.metrics import peak_signal_noise_ratio, mean_squared_error

#   Compute Metrics
mse_val  = mean_squared_error(gray, gray_clahe)
psnr_val = peak_signal_noise_ratio(gray, gray_clahe, data_range=255)

print(f"  MSE  : {mse_val:.4f}")
print(f"  PSNR : {psnr_val:.2f} dB")
```

```
  MSE  : 621.8774
  PSNR : 20.19 dB
```

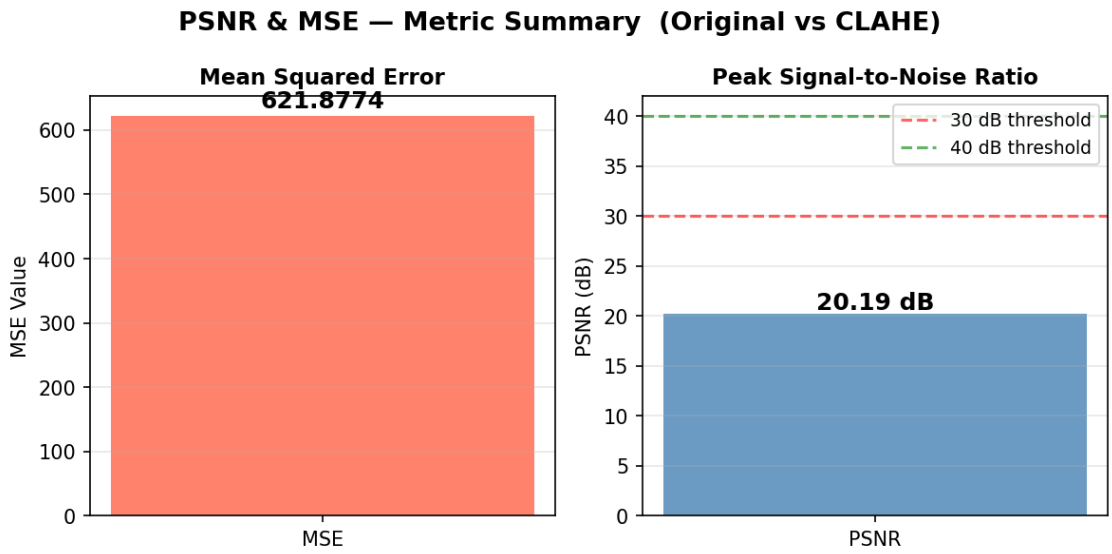
[42]:
```
# Cell 8a - PSNR & MSE : Image Comparison + Difference Map
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Original
axes[0].imshow(gray,       cmap="gray")
axes[0].set_title("Original Grayscale", fontweight="bold")
axes[0].axis("off")

# CLAHE Enhanced
axes[1].imshow(gray_clahe, cmap="gray")
axes[1].set_title("CLAHE Enhanced", fontweight="bold")
axes[1].axis("off")
```

```
# Difference Map
diff = np.abs(gray.astype(np.float32) - gray_clahe.astype(np.float32))
im   = axes[2].imshow(diff, cmap="gray")
axes[2].set_title("Difference Map\n(|Original - CLAHE|)", fontweight="bold")
axes[2].axis("off")
plt.colorbar(im, ax=axes[2], fraction=0.046, pad=0.04)

plt.suptitle(
    f"PSNR & MSE Analysis  ·  MSE = {mse_val:.4f}   |   PSNR = {psnr_val:.2f}␣
 ↪dB",
    fontsize=13, fontweight="bold"
)
plt.tight_layout()
plt.savefig("psnr_mse_analysis.png", dpi=300, bbox_inches="tight")
plt.show()
```



[32]:
```
# Cell 8b - PSNR & MSE : Metric Summary Bar Chart
fig, axes = plt.subplots(1, 2, figsize=(8, 4))

# MSE bar
axes[0].bar(["MSE"], [mse_val], color="tomato", alpha=0.8, width=0.4)
axes[0].set_title("Mean Squared Error", fontweight="bold")
axes[0].set_ylabel("MSE Value")
axes[0].text(0, mse_val + mse_val * 0.02,
             f"{mse_val:.4f}", ha="center", fontsize=12, fontweight="bold")
axes[0].grid(axis="y", alpha=0.3)

# PSNR bar
axes[1].bar(["PSNR"], [psnr_val], color="steelblue", alpha=0.8, width=0.4)
axes[1].set_title("Peak Signal-to-Noise Ratio", fontweight="bold")
axes[1].set_ylabel("PSNR (dB)")
axes[1].text(0, psnr_val + psnr_val * 0.02,
             f"{psnr_val:.2f} dB", ha="center", fontsize=12, fontweight="bold")
```

```
axes[1].axhline(y=30, color="red",   linestyle="--", alpha=0.6, label="30 dB␣
 ↪threshold")
axes[1].axhline(y=40, color="green", linestyle="--", alpha=0.6, label="40 dB␣
 ↪threshold")
axes[1].legend(fontsize=9)
axes[1].grid(axis="y", alpha=0.3)

plt.suptitle("PSNR & MSE - Metric Summary  (Original vs CLAHE)",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("psnr_mse_bars.png", dpi=300, bbox_inches="tight")
plt.show()

#   Interpretation
print("\n  Interpretation                          ")
print(f"  MSE  = {mse_val:.4f}  → {'Low difference' if mse_val < 100 else 'High␣
 ↪difference'} between images")
print(f"  PSNR = {psnr_val:.2f} dB → ", end="")
if   psnr_val >= 40: print("Excellent quality  (  40 dB)")
elif psnr_val >= 30: print("Good quality       (30-40 dB)")
else:                print("Low quality / high distortion  (< 30 dB)")
print("                                  ")
```




```
  Interpretation
  MSE  = 621.8774  → High difference between images
  PSNR = 20.19 dB → Low quality / high distortion  (< 30 dB)
```

[33]:
```
# Cell 9 - Full Pipeline Summary Grid (images only)
all_stages = [
    ("Original",          gray,           "gray"),
    ("CLAHE Enhanced",    gray_clahe,     "gray"),
    ("Heatmap (JET)",     heatmap_rgb,    None  ),
    ("Canny Edges",       edges,          "gray"),
    ("Morph Open+Close",  after_close,    "gray"),
    ("Contours Detected", contour_canvas, None  ),
]

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

for ax, (title, img, cmap) in zip(axes.flat, all_stages):
    ax.imshow(img, cmap=cmap) if cmap else ax.imshow(img)
    ax.set_title(title, fontsize=11, fontweight="bold", pad=8)
    ax.axis("off")

plt.suptitle("Full Pipeline - All Stages", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("full_pipeline_summary.png", dpi=300, bbox_inches="tight")
plt.show()
```